

PYTHON

PCAD – CERTIFIED ASSOCIATE DATE ANALYST WITH PYTHON

CHAPTER 4 – DATA ANALYSIS AND MODELING

CHAPTER 4 – DATA ANALYSIS AND MODELING

Data Analysis with Pandas and NumPy

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Organizar y limpiar datos usando Pandas
  - Objetivos:
    - Filtrar y ordenar datos en tablas.
    - Detectar y gestionar valores faltantes.
    - Corregir inconsistencias en datasets.
    - Preparar datos para análisis posteriores.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Organizar y limpiar datos usando Pandas
  - A. **Filtrar**, ordenar y gestionar valores faltantes o inconsistentes
    - Ejemplo:

```
ventas = pd.DataFrame({
    'producto': ['Laptop', 'Mouse', 'Laptop', 'Teclado'],
    'precio': [1200, 25, 1300, 80],
    'stock': [5, 50, 2, 20]
})

# Filtrar productos con precio mayor a 100
productos_caros = ventas[ventas['precio'] > 100]

print(productos_caros)
```

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Organizar y limpiar datos usando Pandas
  - A. Filtrar, **ordenar** y gestionar valores faltantes o inconsistentes
    - Ejemplo:

```
# Ordenar por precio descendente
ventas_ordenadas = ventas.sort_values(by='precio', ascending=False)
```

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Organizar y limpiar datos usando Pandas
  - A. Filtrar, ordenar y **gestionar valores faltantes** o inconsistentes
    - Detectar valores nulos:

```
print(ventas.isnull())
```

    - Eliminar filas con valores nulos:

```
ventas_limpias = ventas.dropna()
```

    - Reemplazar valores nulos:

```
ventas['stock'] = ventas['stock'].fillna(0)
```

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Organizar y limpiar datos usando Pandas
  - A. Filtrar, ordenar y gestionar valores faltantes o **inconsistentes**

```
import pandas as pd

df = pd.DataFrame({
    'ciudad': ['Madrid', ' madrid ', 'MADRID', 'Barcelona']
})

# Limpiar texto
# strip() elimina espacios
# lower() convierte a minúsculas

df['ciudad'] = df['ciudad'].str.strip().str.lower()

print(df)
```

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Organizar y limpiar datos usando Pandas
  - B. Técnicas fundamentales de limpieza de datos

Técnica	Descripción
dropna()	Elimina valores nulos
fillna()	Sustituye valores faltantes
replace()	Reemplaza valores específicos
duplicated()	Detecta duplicados
drop_duplicates()	Elimina duplicados
astype()	Convierte tipos de datos

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Organizar y limpiar datos usando Pandas
  - B. Técnicas fundamentales de limpieza de datos

```python
import pandas as pd

clientes = pd.DataFrame({
    'nombre': ['Ana', 'Luis', 'Luis', None],
    'edad': [25, None, None, 40],
    'ciudad': ['Madrid ', 'Barcelona', 'Barcelona', 'Sevilla']
})

# Eliminar duplicados
clientes = clientes.drop_duplicates()

# Reemplazar valores nulos
clientes['edad'] = clientes['edad'].fillna(clientes['edad'].mean())

# Limpiar espacios
clientes['ciudad'] = clientes['ciudad'].str.strip()

print(clientes)
```

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Combinar y reformatear conjuntos de datos usando Pandas
  - Objetivos
    - Combinar múltiples datasets.
    - Reestructurar información.
    - Aplicar técnicas avanzadas de manipulación.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Combinar y reformatear conjuntos de datos usando Pandas
  - A. Aplicar técnicas avanzadas de manipulación de datos: combinar (**merge**), unir (join), tablas dinámicas (pivot y pivot table) y reorganizar (reshape).
    - Permite combinar dataframes usando **columnas comunes**.

```
clientes = pd.DataFrame({
    'id_cliente': [1, 2, 3],
    'nombre': ['Ana', 'Luis', 'Marta']
})

ventas = pd.DataFrame({
    'id_cliente': [1, 2, 1],
    'venta': [100, 200, 300]
})

resultado = pd.merge(clientes, ventas, on='id_cliente')
```

Similar a inner join de sql

	id_cliente	nombre	venta
0	1	Ana	100
1	1	Ana	300
2	2	Luis	200

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Combinar y reformatear conjuntos de datos usando Pandas
  - A. Aplicar técnicas avanzadas de manipulación de datos: combinar (**merge**), unir (join), tablas dinámicas (pivot y pivot table) y reorganizar (reshape).

```
pd.merge(clientes, ventas, on='id_cliente', how='left')
```

  - Tipos de merge (argumento **how**).

Tipo	Qué conserva
inner	solo coincidencias
left	todas las filas del df izquierdo
right	todas las filas del df derecho
outer	todas las filas de ambos df

	id_cliente	nombre	venta
0	1	Ana	100.0
1	1	Ana	300.0
2	2	Luis	200.0
3	3	Marta	NaN

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Combinar y reformatear conjuntos de datos usando Pandas
  - A. Aplicar técnicas avanzadas de manipulación de datos: combinar (merge), unir (**join**), tablas dinámicas (pivot y pivot table) y reorganizar (reshape).
    - El método join() combina dataframes usando índices.

```
empleados = pd.DataFrame({
    'nombre': ['Ana', 'Luis'],
    'departamento': ['Ventas', 'Marketing']
}, index=[1, 2])

salarios = pd.DataFrame({
    'salario': [3000, 4000]
}, index=[1, 2])

resultado = empleados.join(salarios)
```

	nombre	departamento	salario
1	Ana	Ventas	3000
2	Luis	Marketing	4000

También admite how

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Combinar y reformatear conjuntos de datos usando Pandas
  - A. Aplicar técnicas avanzadas de manipulación de datos: combinar (merge), unir (join), tablas dinámicas (**pivot y pivot table**) y reorganizar (reshape).
    - Métodos pivot y pivot_table
      - pivot()
        - Solo reorganiza datos
        - No admite duplicados
        - No hace agregaciones
        - Más simple y rápido
      - pivot_table()
        - Reorganiza y además agrega/calcula
        - Sí admite duplicados
        - Más flexible

Exponen doble interfaz: pueden ser métodos de un DataFrame o funciones del módulo pandas

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Combinar y reformatear conjuntos de datos usando Pandas
  - A. Aplicar técnicas avanzadas de manipulación de datos: combinar (merge), unir (join), tablas dinámicas (**pivot y pivot table**) y reorganizar (reshape).
    - Las tablas dinámicas (**pivot table**) resumen información.

```
ventas = pd.DataFrame({
  'region': ['Norte', 'Norte', 'Sur', 'Sur', 'Sur'],
  'producto': ['A', 'B', 'A', 'B', 'B'],
  'ventas': [100, 150, 200, 250, 300]
})

pivot = ventas.pivot_table(
  index='region',
  columns='producto',
  values='ventas',
  aggfunc='sum'
)
```



Sí admite duplicados

producto	A	B
region		
Norte	100	150
Sur	200	550

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Combinar y reformatear conjuntos de datos usando Pandas
  - A. Aplicar técnicas avanzadas de manipulación de datos: combinar (merge), unir (join), tablas dinámicas (**pivot table**) y reorganizar (reshape). Valores de **aggfunc**

Función	Qué hace
'sum'	suma
'mean'	promedio
'median'	mediana
'min'	mínimo
'max'	máximo
'count'	cuenta valores no nulos
'size'	cuenta filas (incluye NaN)
'std'	desviación estándar
'var'	varianza
'prod'	producto
'first'	primer valor
'last'	último valor
'nunique'	número de valores únicos

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Combinar y reformatear conjuntos de datos usando Pandas
  - A. Aplicar técnicas avanzadas de manipulación de datos: combinar (merge), unir (join), tablas dinámicas (**pivot table**) y reorganizar (reshape).
    - El método **pivot** solo reestructura (no es una tabla dinámica):

```
ventas = pd.DataFrame({
    'region': ['Norte', 'Norte', 'Sur', 'Sur'],
    'producto': ['A', 'B', 'A', 'B'],
    'ventas': [100, 150, 200, 250]
})

pivot = ventas.pivot(
    index='region',
    columns='producto',
    values='ventas'
)
```

No admite duplicados

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Combinar y reformatear conjuntos de datos usando Pandas
  - A. Aplicar técnicas avanzadas de manipulación de datos: combinar (merge), unir (join), tablas dinámicas (pivot table) y reorganizar (**reshape**).
    - Reshape con **melt**() → Transforma columnas en filas.

```
ventas = pd.DataFrame({
    'mes': ['Enero', 'Febrero'],
    'producto_A': [100, 150],
    'producto_B': [200, 250]
})

resultado = ventas.melt(id_vars='mes',
                        var_name='producto',
                        value_name='ventas')
```

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Combinar y reformatear conjuntos de datos usando Pandas
  - A. Aplicar técnicas avanzadas de manipulación de datos: combinar (merge), unir (join), tablas dinámicas (**pivot table**) y reorganizar (**reshape**).
    - En pandas, pivot, pivot_table y melt sirven para reorganizar datos:
      - pivot y pivot_table → convierte datos "largos" en formato "ancho"
      - melt → convierte datos "anchos" en formato "largo"

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Combinar y reformatear conjuntos de datos usando Pandas
  - B. Estructurar dataset para análisis específico
    - Buenas prácticas
      - Mantener nombres de columnas claros.
      - Evitar datos duplicados.
      - Mantener formatos consistentes.
      - Utilizar tipos de datos adecuados.
      - Diseñar tablas según el objetivo del análisis.

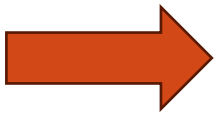
CHAPTER 4 – DATA ANALYSIS AND MODELING

- Relación entre series y dataframes
  - Objetivos
    - Comprender Series y DataFrames.
    - Utilizar indexación y operaciones vectorizadas.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Relación entre series y dataframes
  - A. Diferencias entre Series y DataFrames
    - **Cada columna de un DataFrame es una Series**.
    - Una **Series** es una estructura **1D**.
    - Dispone de índices y datos.

```
serie = pd.Series([10, 20, 30],
                  index=['a', 'b', 'c'])
```



```
a    10
b    20
c    30
dtype: int64
```

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Relación entre series y dataframes
  - A. Diferencias entre Series y DataFrames
    - Un **DataFrame** es una estructura **2D**.
    - Dispone índices, filas y columnas.

```
df = pd.DataFrame({
    'nombre': ['Ana', 'Luis'],
    'edad': [25, 30]
})
```



	nombre	edad
0	Ana	25
1	Luis	30

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Relación entre series y dataframes
  - B. Indexación y funciones vectorizadas
  - Indexación

```
df['nombre']
```

  - Operaciones vectorizadasPandas permite aplicar operaciones a columnas completas.

```
df['edad_doble'] = df['edad'] * 2
```

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Acceso y manipulacíon de datos utilizando localizadores y segmentación.
  - Objetivos
    - Recuperar datos de forma precisa.
    - Modificar registros.
    - Aplicar slicing e indexación.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Acceso y manipulacíon de datos utilizando localizadores y segmentación.
  - A. Uso de **.loc**, .iloc y slicing.
    - .loc→ selección por etiquetas (índices)

```
df.loc['índices']
```

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Acceso y manipulacíon de datos utilizando localizadores y segmentación.
  - A. Uso de .loc, **.iloc** y slicing.
    - .iloc → selección por posición

```
df.iloc[5]
```




```
df = pd.DataFrame({
    'nombre': ['Ana', 'Luis'],
    'edad': [25, 30]
})

df = df.drop(0)
# print(df.loc[0]) # Error
print(df.iloc[0]) # OK -> Posición
print(df.loc[1]) # OK -> Etiqueta
```

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Acceso y manipulacíon de datos utilizando localizadores y segmentación.
  - A. Uso de .loc, .iloc y **slicing**.
    - iloc: slicing por posición (numérico)
      - Funciona exactamente igual que las listas de Python: el límite superior se excluye ([inicio:fin_excluido]).
      - df.iloc[0:3] → Recupera las filas 0,1 y 2.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Acceso y manipulacíon de datos utilizando localizadores y segmentación.
  - A. Uso de .loc, .iloc y **slicing**.
    - iloc: slicing de filas y columnas a la vez
      - La estructura es df.iloc[filas, columnas]
      - df.iloc[0:2, 0:2] → Recupera las filas 0 y1 y las columnas 0 y 1.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Acceso y manipulacíon de datos utilizando localizadores y segmentación.
  - A. Uso de .loc, .iloc y **slicing**.
    - **loc**: slicing de filas por etiquetas
      - La estructura es df.loc[“fila_inicial”:”fila_final”]
      - df.loc['Fila_1':'Fila_3’] → Recupera las filas en el rango ‘fila_inicial’ y ‘fila_final’, ambas incluídas.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Acceso y manipulacíon de datos utilizando localizadores y segmentación.
  - A. Uso de .loc, .iloc y **slicing**.
    - **loc**: slicing de filas y columnas por etiquetas
      - La estructura es df.loc[“fila_inicial”:”fila_final”, “columna_inicial”:”columna_final”]
      - print(df.loc['Fila_1':'Fila_3', 'A':'B'])→ Recupera las filas en el rango ‘fila_inicial’ y ‘fila_final’, ambas incluidas y las columnas entre ‘A’ y ‘B’, ambas incluidas.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Acceso y manipulacíon de datos utilizando localizadores y segmentación.
  - Selección condicional → máscara
    - mascara = df['Ventas'] > 15000
    - empleados_top = df[mascara]
  - Normalmente se compacta el código:
    - empleados_top = df[df[‘Ventas’]>15000]

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Acceso y manipulacíon de datos utilizando localizadores y segmentación.
  - Selección condicional → operadores
    - == (Igual a)
    - != (Diferente de)
    - > , < (Mayor que, menor que)
    - >= , <= (Mayor o igual, menor o igual)

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Acceso y manipulacíon de datos utilizando localizadores y segmentación.
  - Selección condicional → combinaciones → utilizar operadores de bit
    - & (And): Ambas condiciones deben cumplirse.
    - | (Or): Al menos una condición debe cumplirse.
    - ~ (Not): Invierte la condición (lo que era True pasa a False).

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Acceso y manipulacíon de datos utilizando localizadores y segmentación.
  - Selección condicional → métodos especiales.
    - isin() → Coincidencia entre un conjunto

```
# En vez de hacer País == 'España' | País == 'México'
paises_objetivo = ['España', 'México']
df_filtrado = df[df['País'].isin(paises_objetivo)]
```

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Acceso y manipulacíon de datos utilizando localizadores y segmentación.
  - Selección condicional → métodos especiales.
    - between() → Coincidencia por rangos

```
# Filtra empleados con edad entre 25 y 40 años (ambos incluidos)
df_rango_edad = df[df['Edad'].between(25, 40)]
```

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Acceso y manipulacíon de datos utilizando localizadores y segmentación.
  - Selección condicional + selección de columnas.
    - Combinando condición con loc, se obtienen los nombres de las columnas.

```
# Estructura: df.loc[condicion_filas, columnas_a_mostrar]
nombres_top = df.loc[df['Ventas'] > 15000, 'Nombre']
```

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Acceso y manipulacíon de datos utilizando localizadores y segmentación.
  - Modificación de datos
    - Se puede asignar valor al resultado de una selección.

```
df_ventas.loc['Lunes':'Miércoles', 'Ana':'Luis'] = 0
```

	Ana	Luis	María
Lunes	0	0	2
Martes	0	0	4
Miércoles	0	0	3
Jueves	8	4	5
Viernes	10	6	7
Sábado	3	1	2
Domingo	4	2	1

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Acceso y manipulacíon de datos utilizando localizadores y segmentación.
  - B. Estrategias de indexación eficientes
    - Recomendaciones
      - Utilizar índices únicos.
      - Evitar loops cuando existan operaciones vectorizadas.
      - Usar .loc para etiquetas.
      - Usar .iloc para posiciones.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Realizar operaciones con matrices/vectores y distinguir entre las estructuras de datos fundamentales.
  - Objetivos
    - Utilizar NumPy para operaciones con arrays.
    - Comparar estructuras de datos.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Realizar operaciones con matrices/vectores y distinguir entre las estructuras de datos fundamentales.
  - A. Operaciones con NumPy
    - Creación de arrays

```
import numpy as np

arr_1d = np.array([1, 2, 3, 4])

arr_2d = np.array([[1, 2, 3], [4, 5, 6]])

arr_3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

arr_vacio = np.zeros((3, 3))
```

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Realizar operaciones con matrices/vectores y distinguir entre las estructuras de datos fundamentales.
  - A. Operaciones aritméticas con NumPy →
    - Posibles si:
      - **Los dos arrays tienen las mismas dimensiones (número de filas y columnas)**
      - **Uno de ellos es un escalar.**
      - **Si la última dimensión coincide.**

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Realizar operaciones con matrices/vectores y distinguir entre las estructuras de datos fundamentales.
  - A. Operaciones con NumPy → Operaciones de arrays **con escalares**
    - Sumas, restas, multiplicaciones, divisiones, potencias
    - Aplica la operación a cada elemento del array
    - No tienen las mismas dimensiones → se realiza broadcasting → "estirar" el escalar para convertirlo en un array

```
arr + 10
arr * 2
```

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Realizar operaciones con matrices/vectores y distinguir entre las estructuras de datos fundamentales.
  - A. Operaciones con NumPy → Operaciones con matrices
    - Si coincide la última dimensión → **broadcasting** ('estira' hacia abajo la matriz para igualar dimensiones)

```
arr_1d = np.array([1, 2, 3])
arr_2d_2x3 = np.array([[1, 2, 3], [4, 5, 6]])
arr_2d_2x4 = np.array([[1, 2, 3, 4], [4, 5, 6, 7]])
arr_1d * arr_2d_2x3
arr_1d * arr_2d_2x4 # ERROR
```

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Realizar operaciones con matrices/vectores y distinguir entre las estructuras de datos fundamentales.
  - A. Operaciones con NumPy → Ventajas de usar Broadcasting
    - **Eficiencia de memoria**: No se crean copias físicas del arreglo más pequeño en la memoria. Todo el "estiramiento" ocurre a nivel lógico en lenguaje C de alta velocidad.
    - **Código más limpio**: Se evita escribir bucles for para recorrer las filas o columnas, haciendo que el código de Python sea más legible y directo.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Realizar operaciones con matrices/vectores y distinguir entre las estructuras de datos fundamentales.
  - A. Operaciones con NumPy → Agregaciones
  - Las agregaciones en NumPy son funciones que toman un conjunto de datos y lo reducen a un único valor (o a un conjunto más pequeño de valores).
  - Están optimizadas en código compilado, lo que las hace miles de veces más rápidas que usar bucles for tradicionales en Python.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Realizar operaciones con matrices/vectores y distinguir entre las estructuras de datos fundamentales.
  - A. Operaciones con NumPy → Agregaciones
    - Suma total: np.sum(ventas) o ventas.sum()
    - Promedio (Media): np.mean(ventas) o ventas.mean()
    - Valor Mínimo: np.min(ventas) o ventas.min()
    - Valor Máximo: np.max(ventas) o ventas.max()
    - Desviación Estándar: np.std(ventas) o ventas.std()
    - Varianza: np.var(ventas) o ventas.var()
    - Mediana: np.median(ventas)
    - Percentil: np.percentile(ventas, 25)

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Realizar operaciones con matrices/vectores y distinguir entre las estructuras de datos fundamentales.
  - A. Operaciones con NumPy → Agregaciones por filas y columnas (solo matrices)
    - Se debe indicar el eje (axis):
      - axis=0 → devuelve el resultado por cada columna
      - axis=1 → devuelve el resultado por cada fila

```
np.sum(matriz, axis=1))
```

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Realizar operaciones con matrices/vectores y distinguir entre las estructuras de datos fundamentales.
  - A. Operaciones con NumPy → Obtención de posiciones de valores máximos y mínimos
    - Funciones argmin y argmax
    - np.argmin(serie) → Devuelve la posición en la que se encuentra el valor máximo.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Realizar operaciones con matrices/vectores y distinguir entre las estructuras de datos fundamentales.
  - A. Operaciones con NumPy → Agregaciones con valores faltantes
    - np.sum(serie) → si tiene np.nan devuelve nan
    - np.nansum(serie) → ignora np.nan
    - Existen variantes para la mayoría de funciones de agregación.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Realizar operaciones con matrices/vectores y distinguir entre las estructuras de datos fundamentales.
  - B. Diferencias entre estructuras de datos.
  - **Listas** (list). Es la estructura de datos secuencial por excelencia en Python nativo. Su principal característica es la flexibilidad: pueden contener elementos de cualquier tipo (enteros, strings, objetos, e incluso otras listas) y cambiar de tamaño dinámicamente. Esta flexibilidad sacrifica la velocidad.
  - **Arrays** (array.array). Pertenecen al módulo nativo array de Python. A diferencia de las listas, son homogéneos. Son más eficientes en uso de memoria que las listas, pero carecen de funciones avanzadas para análisis matemático o estadístico.
  - **NDArrays** (numpy.ndarray) Es la estructura central de NumPy ("N-Dimensional Array"). Representa bloques de memoria contiguos y homogéneos de N dimensiones. Son la base de la computación científica en Python. Al estar escritos en C, permiten operaciones vectorizadas (como el broadcasting) que eliminan la necesidad de usar bucles for, procesando millones de datos en milisegundos.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Realizar operaciones con matrices/vectores y distinguir entre las estructuras de datos fundamentales.
  - B. Diferencias entre estructuras de datos.
  - **Series** (pandas.Series). Es la estructura unidimensional de Pandas. Cuenta con un índice explícito (etiquetas) que permite acceder a los datos mediante nombres o fechas, en lugar de solo posiciones numéricas. Puede contener datos heterogéneos y maneja de forma nativa los valores faltantes (NaN).
  - **DataFrames** (pandas.DataFrame). Es la estructura bidimensional de Pandas. Un DataFrame es, esencialmente, un contenedor de múltiples Series que comparten el mismo índice. Cada columna puede tener un tipo de datos diferente (números, texto, booleanos), lo que lo convierte en la herramienta estándar para la manipulación y limpieza de datos estructurados.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Realizar operaciones con matrices/vectores y distinguir entre las estructuras de datos fundamentales.
  - B. Diferencias entre estructuras de datos. Rendimiento:
  - **Mayor rapidez:**
    - NDArray (NumPy). Almacena los datos en ubicaciones contiguas de memoria y vectoriza las operaciones. Operaciones como matriz * 2 ocurren casi instantáneamente porque se ejecutan en código C compilado.
  - **Menor consumo de memoria:**
    - array.array y NDArray. Al no guardar punteros individuales ni metadatos de índices (como Pandas), aprovechan cada byte al máximo.
  - **Mayor lentitud:**
    - list (Python). Para operar sobre una lista, Python debe "desempaquetar" cada objeto individualmente (revisar su tipo, buscar su valor), lo que genera un gran cuello de botella (overhead) en volúmenes masivos de datos.
  - **Velocidad y consumo intermedio:**
    - Series y DataFrames. Tienen un consumo de memoria y tiempo de procesamiento ligeramente mayor que NumPy debido a la sobrecarga que generan los índices y las etiquetas, pero la pérdida de milisegundos se compensa con su versatilidad.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Realizar operaciones con matrices/vectores y distinguir entre las estructuras de datos fundamentales.
  - B. Diferencias entre estructuras de datos. Casos de Uso Recomendados
    - **Listas**: Programando de lógica general en Python y manejo de colecciones pequeñas de objetos heterogéneos, o se necesite agregar/quitar elementos constantemente de forma dinámica.
    - **Arrays nativos**: Arrays de números muy largos en Python básico y la memoria de tu entorno sea extremadamente limitada, sin requerir operaciones matemáticas avanzadas.
    - **NDArrays**: Trabajes en computación científica, procesamiento de imágenes, machine learning, o álgebra lineal de alta velocidad.
    - **Series**: Análisis unidimensionales que dependan fuertemente de una etiqueta, como métricas de sensores o precios de acciones financieros ordenados por tiempo (Time Series).
    - **DataFrames**: Ciencia de Datos clásica (limpieza, filtrado, unión de tablas JOIN, agrupaciones groupby). Es ideal para cargar archivos CSV, Excel o consultas SQL y preparar los datos para análisis estadístico.

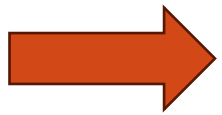
CHAPTER 4 – DATA ANALYSIS AND MODELING

- Agrupar, resumir y extraer información de los datos
  - Objetivos
    - Agrupar y resumir información.
    - Generar estadísticas descriptivas.
    - Detectar patrones y anomalías.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Agrupar, resumir y extraer información de los datos
  - A. **groupby()**, pivot y tablas cruzadas

```
ventas = pd.DataFrame({
    'region': ['Norte', 'Norte', 'Sur', 'Sur'],
    'año': [2020, 2021, 2020, 2021],
    'ventas': [100, 150, 200, 250],
    'compras': [50, 75, 100, 125]
})
```



```
region
Norte    250
Sur      450
Name: ventas, dtype: int64
```

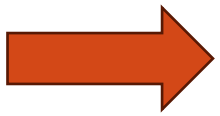
```
resultado = ventas.groupby('region')['ventas'].sum()
```

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Agrupar, resumir y extraer información de los datos
  - A. **groupby()**, pivot y tablas cruzadas

```
ventas = pd.DataFrame({
    'region': ['Norte', 'Norte', 'Sur', 'Sur'],
    'año': [2020, 2021, 2020, 2021],
    'ventas': [100, 150, 200, 250],
    'compras': [50, 75, 100, 125]
})

resultado_compras =
ventas.groupby(['region','año'])['compras'].sum()
```



MultiIndex

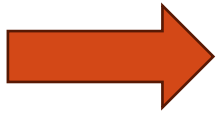
```
region  año
Norte   2020     50
        2021     75
Sur     2020    100
        2021    125
Name: compras, dtype: int64
```

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Agrupar, resumir y extraer información de los datos
  - A. **groupby()**, pivot y tablas cruzadas

```
ventas = pd.DataFrame({
    'region': ['Norte', 'Norte', 'Sur', 'Sur'],
    'ventas': [100, 150, 200, 250],
    'compras': [50, 75, 100, 125]
})

# Aplicamos funciones distintas a cada columna
resultado = ventas.groupby('region').agg({
    'ventas': 'sum',      # Sumamos las ventas
    'compras': 'mean'     # Calculamos el
promedio de las compras
})
```



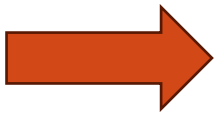
region	ventas	compras
Norte	250	62.5
Sur	450	112.5

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Agrupar, resumir y extraer información de los datos
  - A. groupby(), **pivot** y tablas cruzadas
    - Permite resumir, agrupar y reorganizar un conjunto de datos bidimensional

```
ventas = pd.DataFrame({
    'region': ['Norte', 'Norte', 'Sur', 'Sur'],
    'ventas': [100, 150, 200, 250]
})

pivot = ventas.pivot_table(
    values='ventas',
    index='region',
    aggfunc=['sum', 'mean']
)
```

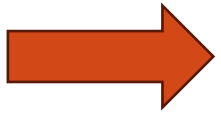


	sum	mean
	ventas	ventas
region		
Norte	250	125.0
Sur	450	225.0

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Agrupar, resumir y extraer información de los datos
  - A. groupby(), pivot y **tablas cruzadas**
    - Las **crosstab** son tablas de frecuencias.

```
encuesta = pd.DataFrame({
    'genero': ['M', 'F', 'F', 'M'],
    'compra': ['Sí', 'Sí', 'No', 'Sí']
})
```



compra	No	Sí
genero		
F	1	1
M	0	2

```
resultado = pd.crosstab(encuesta['genero'],
                        encuesta['compra'])
```

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Agrupar, resumir y extraer información de los datos
  - B Calcular estadísticas descriptivas con Pandas y NumPy para identificar tendencias, detectar anomalías y apoyar la toma de decisiones.

pandas

Método	Descripción	Ejemplo
`describe()`	Genera un resumen estadístico completo del conjunto de datos.	`df['ventas'].describe()`
`count()`	Cuenta valores no nulos.	`df['ventas'].count()`
`sum()`	Calcula la suma de los valores.	`df['ventas'].sum()`
`mean()`	Calcula la media aritmética.	`df['ventas'].mean()`
`median()`	Calcula la mediana.	`df['ventas'].median()`
`mode()`	Obtiene la moda (valor más frecuente).	`df['ventas'].mode()`

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Agrupar, resumir y extraer información de los datos
  - B Calcular estadísticas descriptivas con Pandas y NumPy para identificar tendencias, detectar anomalías y apoyar la toma de decisiones.

Método	Descripción	Ejemplo
`min()`	Devuelve el valor mínimo.	`df['ventas'].min()`
`max()`	Devuelve el valor máximo.	`df['ventas'].max()`
`std()`	Calcula la desviación estándar.	`df['ventas'].std()`
`var()`	Calcula la varianza.	`df['ventas'].var()`
`quantile()`	Obtiene percentiles específicos.	`df['ventas'].quantile(0.25)`
`nunique()`	Cuenta valores únicos.	`df['ventas'].nunique()`
`value_counts()`	Cuenta la frecuencia de cada valor.	`df['producto'].value_counts()`



CHAPTER 4 – DATA ANALYSIS AND MODELING

- Agrupar, resumir y extraer información de los datos
  - B Calcular estadísticas descriptivas con Pandas y NumPy para identificar tendencias, detectar anomalías y apoyar la toma de decisiones.

Método	Descripción	Ejemplo
`corr()`	Calcula la correlación entre variables.	`df.corr()`
`cov()`	Calcula la covarianza entre variables.	`df.cov()`
`skew()`	Mide la asimetría de la distribución.	`df['ventas'].skew()`
`kurt()`	Mide la curtosis (concentración de valores extremos).	`df['ventas'].kurt()`
`idxmin()`	Devuelve el índice del valor mínimo.	`df['ventas'].idxmin()`
`idxmax()`	Devuelve el índice del valor máximo.	`df['ventas'].idxmax()`

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Agrupar, resumir y extraer información de los datos
  - B Calcular estadísticas descriptivas con Pandas y NumPy para identificar tendencias, detectar anomalías y apoyar la toma de decisiones.



Función	Descripción	Ejemplo
`np.mean()`	Calcula la media aritmética.	`np.mean(datos)`
`np.median()`	Calcula la mediana.	`np.median(datos)`
`np.average()`	Media ponderada o simple.	`np.average(datos)`
`np.min()`	Valor mínimo.	`np.min(datos)`
`np.max()`	Valor máximo.	`np.max(datos)`
`np.ptp()`	Rango (máximo - mínimo).	`np.ptp(datos)`

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Agrupar, resumir y extraer información de los datos
  - B Calcular estadísticas descriptivas con Pandas y NumPy para identificar tendencias, detectar anomalías y apoyar la toma de decisiones.



Función	Descripción	Ejemplo
`np.std()`	Desviación estándar.	`np.std(datos)`
`np.var()`	Varianza.	`np.var(datos)`
`np.percentile()`	Calcula percentiles.	`np.percentile(datos, 25)`
`np.quantile()`	Calcula cuantiles.	`np.quantile(datos, 0.25)`
`np.sum()`	Suma total de valores.	`np.sum(datos)`
`np.prod()`	Producto de todos los valores.	`np.prod(datos)`

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Agrupar, resumir y extraer información de los datos
  - B Calcular estadísticas descriptivas con Pandas y NumPy para identificar tendencias, detectar anomalías y apoyar la toma de decisiones.



Función	Descripción	Ejemplo
`np.count_nonzero()`	Cuenta elementos distintos de cero.	`np.count_nonzero(datos)`
`np.unique()`	Devuelve valores únicos.	`np.unique(datos)`
`np.argmax()`	Índice del valor máximo.	`np.argmax(datos)`
`np.argmin()`	Índice del valor mínimo.	`np.argmin(datos)`
`np.corrcoef()`	Calcula matriz de correlación.	`np.corrcoef(x, y)`
`np.cov()`	Calcula matriz de covarianza.	`np.cov(x, y)`

CHAPTER 4 – DATA ANALYSIS AND MODELING

Statistical Methods and Machine Learning

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Aplicar la estadística descriptiva de Python para el análisis de conjuntos de datos.

- A. Calcular e intepretar medidas clave de la estadisítica como media, media, moda, varianza y desviación estándar utilizando Python.

Medida Estadística	Código en Python Puro	Explicación
Media	`media = sum(datos) / len(datos)`	Suma todos los elementos y los divide entre el número total de datos.

Mediana	`datos_ord = sorted(datos)`
	`n = len(datos_ord)`
	`mitad = n // 2`
	`if n % 2 == 0:`
	`  mediana = (datos_ord[mitad - 1] + datos_ord[mitad]) / 2`
	`else:`
	`  mediana = datos_ord[mitad]`
		Ordena los datos. Si el número de elementos es par, promedia los dos centrales; si es impar, toma el del centro.

Moda	`frecuencias = {}`
	`for x in datos:`
	`  frecuencias[x] = frecuencias.get(x, 0) + 1`
	`max_frecuencia = max(frecuencias.values())`
	`moda = [k for k, v in frecuencias.items() if v == max_frecuencia]`
		Cuenta la frecuencia de cada elemento y devuelve el (o los) valor(es) que más se repite(n).

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Aplicar la estadística descriptiva de Python para el análisis de conjuntos de datos.
- A. Calcular e intepretar medidas clave de la estadisítica como media, media, moda, varianza y desviación estándar utilizando Python.

Medida Estadística	Código en Python Puro	Explicación Breve
Varianza	`media = sum(datos) / len(datos)` `varianza = sum((x - media) ** 2 for x in datos) / len(datos)`	Calcula el promedio de las diferencias al cuadrado respecto a la media.

Desviación Estándar	`media = sum(datos) / len(datos)`
	`varianza = sum((x - media) ** 2 for x in datos) / len(datos)`
	`desviacion = varianza ** 0.5`
		Es la raíz cuadrada de la varianza (calculada aquí elevando a 0.5).

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Aplicar la estadística descriptiva de Python para el análisis de conjuntos de datos.
- B. Utilizar librerías Python (como Pandas y Numpy) para generar y analizar estadística descriptiva en conjuntos de datos del mundo real.

Medida	Interpretación	Cálculo en Pandas	Cálculo en NumPy
Media	El "centro de gravedad" o promedio. Muy sensible a valores extremos.	df['col'].mean()	np.mean(arr)
Mediana	El dato del centro (percentil 50). No le afectan los valores atípicos.	df['col'].median()	np.median(arr)
Moda	El valor que más veces se repite en el conjunto de datos.	df['col'].mode()[0]	No dispone de función específica.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Aplicar la estadística descriptiva de Python para el análisis de conjuntos de datos.
- B. Utilizar librerías Python (como Pandas y Numpy) para generar y analizar estadística descriptiva en conjuntos de datos del mundo real.

Medida	Interpretación	Cálculo en Pandas	Cálculo en Numpy
Varianza	Dispersión de los datos respecto a la media (en unidades al cuadrado).	df['col'].var()	np.var(arr)
Desviación Estándar	Cuánto se alejan los datos del promedio en promedio (mismas unidades que los datos).	df['col'].std()	np.std(arr)

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Reconocer la importancia de los conjuntos de prueba (test datasets) en la evaluación de modelos.
- A. Comprender el papel de los conjuntos de datos de prueba en la validación del rendimiento de los modelos de aprendizaje automático (machine learning).
- En Machine Learning, los datos suelen dividirse en:
  - Training Set (entrenamiento): utilizado para aprender patrones.
  - Test Set (prueba): utilizado para evaluar el rendimiento final.
- ¿Por qué es importante?
  - Si evaluamos un modelo utilizando los mismos datos con los que fue entrenado, los resultados serán demasiado optimistas.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Reconocer la importancia de los conjuntos de prueba (test datasets) en la evaluación de modelos.
- B. Demostrar conocimiento sobre la selección y el uso adecuados de conjuntos de datos de prueba para garantizar una evaluación del modelo precisa e imparcial.
- Buenas prácticas
  - 1. Mantener separados entrenamiento y prueba. Nunca utilizar los datos de prueba durante el entrenamiento.
  - 2. Utilizar una muestra representativa. El conjunto de prueba debe reflejar las características del conjunto completo.
  - 3. Evitar fuga de datos (**data leak** o **data breach**). La fuga de datos ocurre cuando información del conjunto de prueba llega al entrenamiento.
  - 4. Utilizar validación cruzada cuando sea necesario. La validación cruzada divide los datos varias veces para obtener una evaluación más robusta.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Analizar y evaluar algoritmos de aprendizaje supervisado y la precisión de los modelos
- A. Analizar diversos algoritmos de aprendizaje supervisado para comprender sus características específicas y sus aplicaciones.
  - Regresión Lineal
    - Predice valores numéricos continuos.
    - Ejemplo:
      - Predecir el precio de una vivienda según:
        - Metros cuadrados.
        - Número de habitaciones.
        - Ubicación.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Analizar y evaluar algoritmos de aprendizaje supervisado y la precisión de los modelos
- A. Analizar diversos algoritmos de aprendizaje supervisado para comprender sus características específicas y sus aplicaciones.
  - Regresión Logística
    - Predice probabilidades y clasificaciones.
    - Ejemplo:
      - Determinar si un correo es:
        - Spam.
        - No spam.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Analizar y evaluar algoritmos de aprendizaje supervisado y la precisión de los modelos
- A. Analizar diversos algoritmos de aprendizaje supervisado para comprender sus características específicas y sus aplicaciones.
  - Árboles de Decisión
    - Crean reglas de decisión en forma de árbol.
    - Ejemplo:
      - Aprobar o rechazar una solicitud de crédito.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Analizar y evaluar algoritmos de aprendizaje supervisado y la precisión de los modelos
- A. Analizar diversos algoritmos de aprendizaje supervisado para comprender sus características específicas y sus aplicaciones.
  - Random Forest
    - Combina múltiples árboles de decisión.
    - Ventajas:
      - Mayor precisión.
      - Menor riesgo de sobreajuste.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Analizar y evaluar algoritmos de aprendizaje supervisado y la precisión de los modelos
- A. Analizar diversos algoritmos de aprendizaje supervisado para comprender sus características específicas y sus aplicaciones.
  - K-Nearest Neighbors (KNN)
    - Clasifica según los vecinos más cercanos.
    - Ejemplo:
      - Recomendación de productos similares.

CHAPTER 4 – DATA ANALYSIS AND MODELING

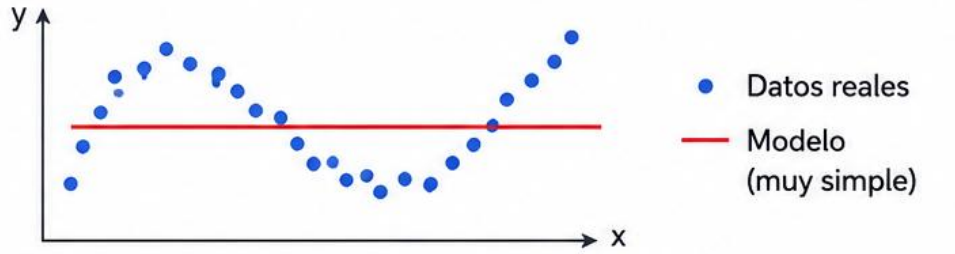
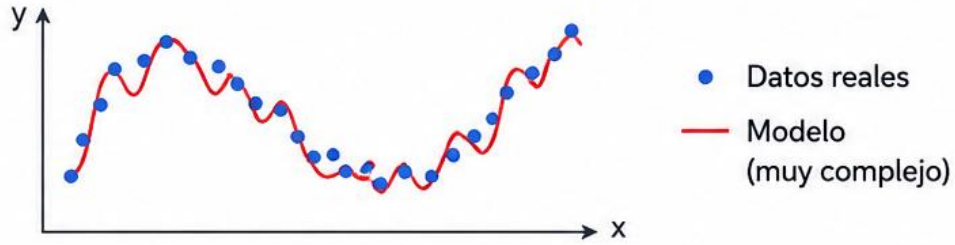
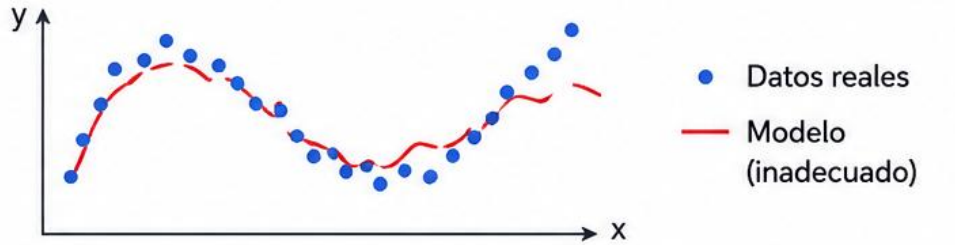
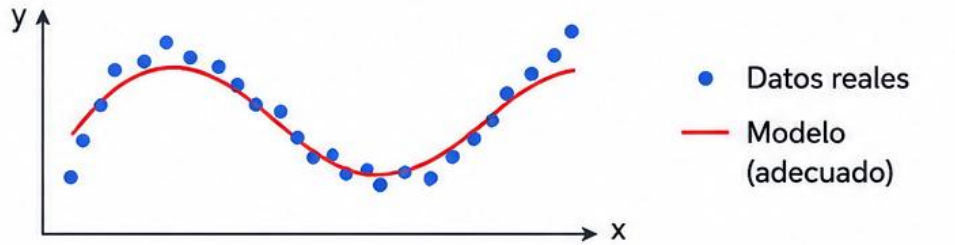
- Analizar y evaluar algoritmos de aprendizaje supervisado y la precisión de los modelos
- B. Evaluar los conceptos de sobreajuste (overfitting) y subajuste (underfitting) en estos modelos, incluyendo una explicación detallada del equilibrio entre sesgo y varianza (bias-variance tradeoff).
- Underfitting (Subajuste) → Ocurre cuando el modelo es demasiado simple.
  - Características:
    - Bajo rendimiento en entrenamiento.
    - Bajo rendimiento en prueba.
- Overfitting (Sobreajuste)
  - Ocurre cuando el modelo aprende demasiado los datos de entrenamiento.
  - Características:
    - Excelente rendimiento en entrenamiento.
    - Mal rendimiento en prueba.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Analizar y evaluar algoritmos de aprendizaje supervisado y la precisión de los modelos
- B. Evaluar los conceptos de sobreajuste (overfitting) y subajuste (underfitting) en estos modelos, incluyendo una explicación detallada del equilibrio entre sesgo y varianza (bias-variance tradeoff).
- Bias-Variance Tradeoff. Existe un equilibrio entre:
  - Bias (Sesgo) → Error producido por suposiciones demasiado simples. Cuánto de lejos (en promedio) están las predicciones respecto de la realidad.
    - Modelos simples.
    - Riesgo de underfitting.
  - Variance (Varianza) → Sensibilidad excesiva a los datos de entrenamiento. Cuánto cambia el modelo si se cambian los datos de entrenamiento.
    - Modelos complejos.
    - Riesgo de overfitting.

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Analizar y evaluar algoritmos de aprendizaje supervisado y la precisión de los modelos
- B. Evaluar los conceptos de sobreajuste (overfitting) y subajuste (underfitting) en estos modelos, incluyendo una explicación detallada del equilibrio entre sesgo y varianza (bias-variance tradeoff).
- Bias-Variance Tradeoff. Existe un equilibrio entre:
  - Objetivo
    - Encontrar un equilibrio donde:
      - El sesgo sea razonablemente bajo.
      - La varianza sea razonablemente baja.
    - Modelo muy simple → Alto Bias, Baja Variance → Underfitting
    - Modelo muy complejo → Bajo Bias, Alta Variance → Overfitting

Bias	Variance	Ilustración	Resultado
Alto	Bajo		**Underfitting** El modelo es demasiado simple y no captura el patrón de los datos.
Bajo	Alto		**Overfitting** El modelo se ajusta demasiado a los datos de entrenamiento, incluido el ruido.
Alto	Alto		**Mal modelo** El modelo es demasiado simple para capturar los patrones y además varía demasiado.
Bajo	Bajo		**Situación deseada** El modelo captura el patrón real sin ajustarse al ruido y generaliza bien.

Idea clave: Buscamos un equilibrio: bajo bias (para aprender bien) y baja variance (para generalizar bien).

CHAPTER 4 – DATA ANALYSIS AND MODELING

- Analizar y evaluar algoritmos de aprendizaje supervisado y la precisión de los modelos
- C. Evaluar las tendencias inherentes de la regresión lineal y la regresión logística en relación con este equilibrio entre sesgo y varianza, y aplicar esta comprensión para prevenir problemas de precisión o rendimiento en los modelos.

Algoritmo	Bias	Variance
Regresión lineal	Alto (si el problema NO es lineal)	Bajo
Regresión logística	Medio-Alto	Bajo
Árbol de decisión	Bajo	Alto
Random Forest	Bajo-Medio	Bajo-Medio
K-Nearest Neighbors (KNN)	Depende de K (K pequeño → Bias pequeño)	Depende de K (K pequeño → Variance alta)